Because balance is protected, it's available only within Account objects.

Variables

Now that we've gone to the trouble to create all these objects, let's make sure we don't lose them. Variables are used to keep track of objects; each variable holds a reference to an object. Let's confirm this with some code:

```
person = "Tim"
puts "The object in 'person' is a #{person.class}"
puts "The object has an id of #{person.object_id}"
puts "and a value of '#{person}'"
```

produces:

```
The object in 'person' is a String
The object has an id of 60
and a value of 'Tim'
```

On the first line, Ruby creates a new string object with the value Tim. A reference to this object is placed in the local variable person. A quick check shows that the variable has indeed taken on the personality of a string, with a class, an object id, and a value.

So, is a variable an object? In Ruby, the answer is "no." A variable is simply a reference to an object. Objects float around in a big pool somewhere (the heap, most of the time) and are pointed to by variables. Let's make the example slightly more complicated:

```
person1 = "Tim"
person2 = person1
person1[0] = 'J'

puts "person1 is #{person1}"
puts "person2 is #{person2}"
```

produces:

```
person1 is Jim
person2 is Jim
```

What happened here? We changed the first character of person1 (Ruby strings are mutable, unlike Java), but both person1 and person2 changed from Tim to Jim.

It all comes back to the fact that variables hold references to objects, not the objects themselves. Assigning person1 to person2 doesn't create any new objects; it simply copies person1's object reference to person2 so that both person1 and person2 refer to the same object, as shown in the following illustration.